

Dokumentacja – Parcel Scanner Sim API Testing

Spis treści

1. Opis projektu	3
2. Informacje techniczne API	3
3. Lista endpointów	3
4. Strategia testowania API	4
5. Przypadki testowe.....	5
6. Wykonywanie testów.....	10
7. Defekty i naprawa.....	10
8. Testowanie wydajnościowe	13

1. Opis projektu

REST API stworzone lokalnie z wykorzystaniem lekkiego i asynchronicznego frameworka **Ktor**. Zastąpił on do tej pory wykorzystywany **Postman MockServer** który sprawiał kłopoty w wykorzystaniu z metodami innymi niż GET.

2. Informacje techniczne API

Uruchamiany lokalnie tylko w celach testowych z wykorzystaniem `base_url` localhosta i portu 8080. Wykorzystany format danych: **JSON**. Autoryzacja do endpointów logowania wykorzystuje prosty **Bearer Token (JWT)**.

3. Lista endpointów

Endpoint	Metoda	Autoryzacja
/auth/login	POST	Tak
/history/packages	GET	Tak
/history/packages	POST	Tak
/history/forms	GET	Tak
/history/forms	POST	Tak
/auth/logout	POST	Nie

- **/auth/login**

Endpoint służący do logowania w naszej aplikacji, jest on wykorzystywany przy odbijaniu karty w systemie NFC logon, sprawdza w zapisanych wartościach czy ID znajdujące się na karcie istnieje w systemie, jeżeli tak, to zwraca body typu JSON w którym znajdują się dane do dalszej obsługi w aplikacji (typu dane do sesji pracownika, czas rozpoczęcia zmiany, token, status)

- **/history/packages**

Dla metody **GET** używamy sprawdzenia zalogowanego pracownika (jego ID) a następnie odpowiednio uzupełniamy body JSON o listę paczek dopasowanych dla niego. Dla **POST** także, tym razem w celu dodania paczki do listy.

- **/history/forms**

Dla metody **GET** używamy sprawdzenia zalogowanego pracownika (jego ID) a następnie odpowiednio uzupełniamy body JSON o listę formularzy i dokumentów dopasowanych dla niego. Dla **POST** także, tym razem w celu dodania formularza lub dokumentu do listy.

- **/auth/logout**

Metoda służąca do wylogowania się w aplikacji, czyści dane sesji, zwraca response z kodem 200.

4. Strategia testowania API

Z wykorzystaniem niżej stworzonego scenariusza testów odwzorujemy zapytania do API, aby zweryfikować poprawność zwróconych kodów statusu oraz poprawność danych, sprawdzimy także, co stanie się przy użyciu nieprawidłowych oraz danych przybrzeżnych. W planach także sprawdzimy wydajność zapytań przy użyciu narzędzi do testowania wydajnościowego.

Autoryzacja –

TC-A01 : Logowanie poprawnym UID

TC-A02 : Logowanie nieistniejącym UID

TC-A03 : Logowanie z pustym body

TC-A04 : Poprawne wylogowanie użytkownika

Historia (Paczki) –

TC-H01 : Pobieranie historii paczek dla zalogowanego użytkownika

TC-H02 : Pobieranie historii paczek dla niepoprawnego ID

TC-H03 : Pobieranie historii paczek bez podania ID

TC-H04 : Dodanie nowej paczki z poprawnymi wartościami

TC-H05 : Dodanie nowej paczki z niewłaściwymi wartościami

TC-H06 : Dodanie nowej paczki bez podawania żadnych wartości

Historia (Formularze) –

TC-H07 : Pobieranie historii formularzy dla zalogowanego użytkownika

TC-H08 : Pobieranie historii formularzy dla niepoprawnego ID

TC-H09 : Pobieranie historii formularzy bez podania ID

TC-H10 : Dodanie nowego formularza z poprawnymi wartościami

TC-H11 : Dodanie nowego formularza z niewłaściwymi wartościami

TC-H12 : Dodanie nowego formularza bez podawania żadnych wartości

5. Przypadki testowe

Identyfikator	TC-A01
Priorytet	Wysoki
Opis	Weryfikacja logowania z wykorzystaniem poprawnego identyfikatora istniejącego w systemie
Dane wejściowe	Metoda: POST Body: {"cardUid": "DEV MODE UID 123"}
Kroki	1. Wyślij zapytanie POST na /auth/login
Oczekiwany rezultat	Status code: 200 OK. Body zawiera obiekt użytkownika User z danymi Jana Kowalskiego. Body zawiera obiekt Session z wygenerowanymi sessionID. Czas odpowiedzi zapytania < 500ms.

Identyfikator	TC-A02
Priorytet	Wysoki
Opis	Weryfikacja logowania z wykorzystaniem nieistniejącego identyfikatora w systemie
Dane wejściowe	Metoda: POST Body: {"cardUid": "FAKE UID 123444"}
Kroki	1. Wyślij zapytanie POST na /auth/login
Oczekiwany rezultat	Status Code: 404 Not Found Body zawiera komunikat błędu adekwatny do zapytania Czas odpowiedzi zapytania < 500ms.

Identyfikator	TC-A03
Priorytet	Średni
Opis	Weryfikacja logowania bez podawania wartości w body
Dane wejściowe	Metoda: POST Body: {}
Kroki	1. Wyślij zapytanie POST na /auth/login
Oczekiwany rezultat	Status code: 400 Bad Request

Identyfikator	TC-A04
Priorytet	Średni
Opis	Weryfikacja poprawności wylogowania użytkownika
Dane wejściowe	Metoda: POST Body: -
Kroki	1. Wyślij zapytanie POST na /auth/logout
Oczekiwany rezultat	Status code: 200 OK.

	Body zawiera informacje o poprawnym wylogowaniu.
--	--

Identyfikator	TC-H01
Priorytet	Wysoki
Opis	Weryfikacja pobierania listy paczek przypisaną do konkretnego zalogowanego użytkownika istniejącego w systemie
Dane wejściowe	Metoda: GET Header: X-User-ID: EMP-001
Kroki	1. Wyślij zapytanie GET na /history/packages
Oczekiwany rezultat	Status code: 200 OK. Body zawiera tablice obiektów typu Package. Zwrócona lista paczek należy do przypisanego użytkownika. Czas odpowiedzi zapytania < 1500ms.

Identyfikator	TC-H02
Priorytet	Średni
Opis	Weryfikacja pobierania listy paczek dla użytkownika podającego niepoprawne ID (nieistniejące)
Dane wejściowe	Metoda: GET Header: X-User-ID: TEST-9901
Kroki	1. Wyślij zapytanie GET na /history/packages
Oczekiwany rezultat	Status code: 404 Not Found (lub 401 Unauthorized)

Identyfikator	TC-H03
Priorytet	Średni
Opis	Weryfikacja pobierania listy paczek bez podawania ID
Dane wejściowe	Metoda: GET Header: -
Kroki	1. Wyślij zapytanie GET na /history/packages
Oczekiwany rezultat	Status code: 401 Unauthorized Body odpowiedzi zwraca odpowiednią wiadomość błędu

Identyfikator	TC-H04
Priorytet	Wysoki
Opis	Weryfikacja poprawnego zapisu zeskanowanej paczki w bazie
Dane wejściowe	Metoda: POST Header: X-User-ID: EMP-001 Body: { "trackingNumber": "TESTOWYADD-009", "courier": "INPOST", "scanDate": "2025-11-05", "weightClass": "A", "shipmentType": "DOCUMENTS" }
Kroki	1. Wyślij zapytanie POST na /history/packages
Oczekiwany rezultat	Status code: 201 Created.

	Zwrócony obiekt w body zawiera poprawnie wprowadzone dane wysłanej paczki. Paczka widoczna na liście po ponownym pobraniu.
--	---

Identyfikator	TC-H05
Priorytet	Średni
Opis	Weryfikacja zapisu zeskanowanej paczki z podaniem danych w nieprawidłowym formacie
Dane wejściowe	Metoda: POST Header: X-User-ID: EMP-001 Body: <pre>{ "trackingNumber": "TESTOWYADD-009", "courier": null, "scanDate": "2025-11-05", "weightClass": 9, "shipmentType": "DOCUMENTS" }</pre>
Kroki	1. Wyślij zapytanie POST na /history/packages
Oczekiwany rezultat	Status code: 400 Bad Request Odpowiedź powinna zawierać kod błędu

Identyfikator	TC-H06
Priorytet	Średni
Opis	Weryfikacja zapisu zeskanowanej paczki bez podawania danych
Dane wejściowe	Metoda: POST Header: -
Kroki	1. Wyślij zapytanie POST na /history/packages
Oczekiwany rezultat	Status code: 401 Unauthorized

Identyfikator	TC-H07
Priorytet	Wysoki
Opis	Weryfikacja pobierania listy formularzy przypisaną do konkretnego zalogowanego użytkownika istniejącego w systemie
Dane wejściowe	Metoda: GET Header: X-User-ID: EMP001
Kroki	1. Wyślij zapytanie GET na /history/forms
Oczekiwany rezultat	Status code: 200 OK. Body zawiera tablice obiektów typu Form. Zwrócona lista formularzy należy do przypisanego użytkownika. Czas odpowiedzi zapytania < 1500ms.

Identyfikator	TC-H08
Priorytet	Średni
Opis	Weryfikacja pobierania listy formularzy dla użytkownika podającego niepoprawne ID (nieistniejące)
Dane wejściowe	Metoda: GET Header: X-User-ID: TEST9901

Kroki	1. Wyślij zapytanie GET na /history/forms
Oczekiwany rezultat	Status code: 401 Unauthorized

Identyfikator	TC-H09
Priorytet	Wysoki
Opis	Weryfikacja pobrania listy formularzy bez podawania ID
Dane wejściowe	Metoda: GET Header: -
Kroki	1. Wyślij zapytanie GET na /history/forms
Oczekiwany rezultat	Status code: 401 Unauthorized

Identyfikator	TC-H10
Priorytet	Wysoki
Opis	Weryfikacja zapisu poprawnie zeskanowanego formularza w bazie
Dane wejściowe	Metoda: POST Header: X-User-ID: EMP-001 Body: <pre>{ "firstName": "Józef", "lastName": "Testowy", "date": "2025-10-27", "id": "TESTFORM-001", "address": null, "phoneNumber": null, "email": null, "additionalNotes": "testowy POST formularza" }</pre>
Kroki	1. Wyślij zapytanie POST na /history/forms
Oczekiwany rezultat	Status code: 201 Created. Zwrócony obiekt w body zawiera poprawnie wprowadzone dane wysłanego formularza. Formularz widoczny na liście po ponownym pobraniu.

Identyfikator	TC-H11
Priorytet	Średni
Opis	Weryfikacja zapisu formularza podając dane w nieprawidłowym formacie
Dane wejściowe	Metoda: POST Header: X-User-ID: EMP-001 Body: <pre>{ "firstName": null, "lastName": "Testowy", "date": "2025-10-27", "id": "TESTFORM-001", "address": null, "phoneNumber": 12345, "email": false, "additionalNotes": false }</pre>

Kroki	1. Wyślij zapytanie POST na /history/forms
Oczekiwany rezultat	Status code: 400 Bad Request Odpowiedź powinna zawierać kod błędu

Identyfikator	TC-H12
Priorytet	Wysoki
Opis	Weryfikacja zapisu formularza bez podania danych
Dane wejściowe	Metoda: POST Header: - Body: { }
Kroki	1. Wyślij zapytanie POST na /history/forms
Oczekiwany rezultat	Status code: 401 Unauthorized

6. Wykonywanie testów

Parcel Scanner Sim - API Te... - Run results						
Ran today at 02:35:58 PM · View all runs						
Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	Parcel Scanner Sim	1	1s 448ms	32	0	3 ms
RUN SUMMARY						
						1
> POST	[POST] Login method without entering any credentials	2	0			
> POST	[POST] Login method after entering wrong credentials	3	0			
> POST	[POST] Login method after entering existing credentials	4	0			
> POST	Logout method	2	0			
✓ GET	[GET] Get packages history for non-existing user	0	1		×	
FAIL	Verify if adding non existing user ID returns 401 status code				×	
> GET	[GET] Get packages history without entering ID	2	0			
> GET	[GET] Get packages history for existing user with correct ID	3	0			
> POST	[POST] Add package with correctly entered informations	2	0			
> POST	[POST] Add package with wrong types entered into body	2	0			
> POST	[POST] Add package without informations and ID	1	0			
> GET	[GET] Get forms history for existing user with correct ID	3	0			
> GET	[GET] Get forms history for user without entering ID	2	0			
✓ GET	[GET] Get forms history for non-existing user	0	1		×	
FAIL	Verify if adding non existing user ID returns 401 status code				×	
> POST	[POST] Add form with correct informations	2	0			
> POST	[POST] Add form with wrong types entered into body	2	0			
POST	[POST] Add form without entering body informations	0	0			

Podczas uruchomienia zestawu testów dla każdego przypadku możemy zauważyć, że napotkaliśmy dwa testy które nie przeszły pomyślnie, a więc po wstępnej analizie utworzone zostały dwa zgłoszenia z błędami napotkanymi w API.

7. Defekty i naprawa

Tytuł	[BUG-01] Nieobsłużony error podczas pobierania listy przesyłek dla nieistniejącego użytkownika
Opis	Podczas pobierania listy przesyłek podając identyfikator w poprawnym formacie, ale dla nieistniejącego użytkownika zamiast błędu otrzymujemy pustą tablicę.

Kroki do wykonania	1. W zapytaniu GET dla endpoint /history/packages ustaw Header X-User-ID: wartością NULL-9901 2. Wyślij zapytanie
Oczekiwany rezultat	Zwrócona odpowiedź z API powinna zawierać kod błędu 401 wraz z odpowiednim komunikatem błędu
Rzeczywisty rezultat	Zwrócona odpowiedź z API ma status 200 OK oraz zwrócone body zawiera pustą tablicę JSON



Tytuł	[BUG-02] Nieobsłużony error podczas pobierania listy formularzy dla nieistniejącego użytkownika
Opis	Podczas pobierania listy dokumentów podając identyfikator w poprawnym formacie ale dla nieistniejącego użytkownika zamiast błędu otrzymujemy pustą tablicę.
Kroki do wykonania	1. W zapytaniu GET dla endpoint /history/forms ustaw Header X-User-ID: wartością TEST-000991 2. Wyślij zapytanie
Oczekiwany rezultat	Zwrócona odpowiedź z API powinna zawierać kod błędu 401 wraz z odpowiednim komunikatem błędu
Rzeczywisty rezultat	Zwrócona odpowiedź z API ma status 200 OK oraz zwrócone body zawiera pustą tablicę JSON



Rozwiązanie tych błędów polegało na naprawie warunku sprawdzającego poprawność wprowadzonego id / weryfikacji czy użytkownik istnieje w liście zaufanych identyfikatorów w bazie.

```
// === ENDPOINT: HISTORIA PACZEK (GET) ===
get( path = "/history/packages" ) {

    //GET USER ID
    val userId = call.request.headers["X-User-ID"]

    if (userId == null) {
        call.respond( status = HttpStatusCode.Unauthorized, message = mapOf("error" to "Missing User ID"))
        return@get
    }

    val userPackages = Database.packagesHistory[userId] ?: emptyList()
    call.respond( status = HttpStatusCode.OK, message = userPackages)
}
```

Przedstawiony kawałek kodu to kod endpointu zwracającego listę paczek, jak widać sprawdzamy tutaj tylko czy `userId` jest puste, czyli jest `null`, jeżeli będzie błędne to ten warunek zostanie ominięty a jest to błąd.

```
val userExists = Database.authorizedUsers.values.any { it.id == userId }

if (!userExists) {
    call.respond(HttpStatusCode.Unauthorized, mapOf("error" to "User not
authorized"))
    return@get
}
```

Rozwiązaniem było odpowiednie sprawdzenie czy użytkownik znajduje się w liście zaufanych identyfikatorów i odpowiednie obsłużenie zapytania. Taki sam warunek został dodany dla drugiego zgłoszonego błędu w zapytaniu do pobierania listy formularzy. Wynikiem naprawy jest pomyślny proces uruchomienia testów co wynika z przedstawionego zrzutu ekranu:

 Ran today at 03:48:54 PM · [View all runs](#)

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	Parcel Scanner Sim	1	2s 545ms	33	0	20 ms

8. Testowanie wydajnościowe

W planie testów testowanie wydajnościowe podzieliłem na dwie grupy, testowanie wydajnościowe aplikacji jako całości oraz testowanie wydajnościowe API w celu zbadania jak endpointy będą sobie radziły z dużą ilością użytkowników oraz różnymi rodzajami sieci i opóźnień.

Przypadki testowe –

ID Testu	Opis scenariusza	Liczba użytkowników	Ramp-up period	Pętla	Cele weryfikacji
PT-01	Pobieranie listy paczek pod standardowym obciążeniem	15 użytkowników	5 sekund	10	Sprawdzenie czy pobieranie działa przy typowym scenariuszu użycia API
PT-02	Pobieranie listy paczek przy wzmożonym ruchu	50 użytkowników	10 sekund	20	Sprawdzenie pobierania listy paczek przy wzmożonym ruchu
PT-03	Pobieranie listy paczek dla dużej liczby użytkowników	125 użytkowników	3 sekundy	50	Sprawdzenie jak system zachowa się podczas bardzo dużego ruchu podczas pobierania listy paczek
PT-04	Dostęp do nieautoryzowanych zasobów (lista paczek)	100 użytkowników	1 sekunda	10	Sprawdzenie czy mechanizmy bezpieczeństwa działają odpowiednio przy obciążeniu.
PT-05	Pobieranie listy formularzy pod standardowym obciążeniem	15 użytkowników	5 sekund	10	Czy pobieranie listy formularzy działa przy użyciu API
PT-06	Pobieranie listy formularzy przy wzmożonym ruchu	50 użytkowników	10 sekund	20	Sprawdzenie pobierania listy formularzy przy wzmożonym ruchu
PT-07	Pobieranie listy formularzy przy dużej liczbie użytkowników	125 użytkowników	3 sekundy	50	Sprawdzenie jak zachowa się system przy dużej liczbie ruchu podczas pobierania listy formularzy
PT-08	Dodawanie bardzo dużej ilości paczek w krótkim czasie	50	5 sekund	10	Sprawdzenie czy dodanie dużej liczby paczek w krótkim czasie przebiega bezbłędnie.

PT-09	Dodawanie bardzo dużej liczby formularzy w krótkim czasie	50	5 sekund	10	Sprawdzenie czy dodanie dużej liczby formularzy w krótkim czasie przebiega bezbłędnie.
PT-10	Dodanie paczki z nieprawidłowymi typami pól przez dużą ilość użytkowników	35	2 sekundy	10	Sprawdzenie czy mechanizm sprawdzania poprawności pól w metodach POST poradzi sobie przy dużej ilości użytkowników

Niżej zamieszczam tabele z pierwszych testów wydajnościowych naszego API, można zauważyć, że uzyskaliśmy dość akceptowalny czas odpowiedzi z naszego backendu, throughput dla metod POST również nie jest niepokojący, lecz w niektórych grupach testów można zauważyć, że procent testów oznaczonych niepowodzeniem oscyluje w wartościach 5-19% co przykuło moją uwagę.

Label	# Samples ↑	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
[GET] Get package list under small network traffic	96	39	1	192	40.38	20.83%	17.6/sec	4615.95	2.61	268488.0
[GET] Get forms list under small network traffic	96	48	1	471	63.83	20.83%	17.5/sec	5769.65	2.55	337193.8
[POST] Adding packages with wrong types of data in short time	350	13	0	183	20.79	100.00%	149.8/sec	36.56	51.45	250.0
[GET] Get forms list under higher network traffic	460	220	0	44457	2665.51	15.87%	6.9/sec	1954.15	1.02	291634.9
[GET] Get package list under higher network traffic	469	454	0	47407	3826.92	14.29%	7.0/sec	2034.60	1.03	297824.3
[POST] Correctly add form under higher network traffic	483	36	0	4125	191.29	0.00%	8.2/sec	2.63	3.32	329.6
[POST] Correctly add package under higher network traffic	486	47	0	2940	190.62	0.00%	58.8/sec	17.09	21.05	297.8
[GET] Try to get package list as unauthorized user	3849	50	0	31835	595.57	100.00%	57.4/sec	11.72	8.69	209.0
[GET] Get form list under stress testing army group	3869	207	1	49772	2592.86	15.33%	57.7/sec	16248.69	8.53	288237.2
[GET] Get package list under stress testing army group	3902	181	0	49590	2327.95	17.35%	58.2/sec	15977.04	8.62	281025.6
TOTAL	14060	147	0	49772	2045.29	40.18%	209.7/sec	37065.49	35.62	181004.6

Analizując logi z Ktor możemy zauważyć, że większość tych błędów posiada status kodu 500 Internal Server Error a IntelliJ rzuca wyjątkiem typu **ConcurrentModificationException**.

Tytuł	[BUG-PERF-01] Wystąpienie Race Condition przy jednoczesnym dostępie do danych metodą GET
Opis	Podczas testów obciążeniowych (Stress Test, 125 użytkowników) zaobserwowano błędy HTTP 500 Internal Server Error dla 3-10% zapytań typu GET.
Kroki do wykonania	1. Otwórz program JMeter z załadowanym scenariuszem testów 2. Uruchom wszystkie testy dla 10,50 i 125 użytkowników. 3. W ostatniej rubryce wybierz Summary Report
Oczekiwany rezultat	Wszystkie testy wydajnościowe mają procent Error mniejszy niż 0.2%, oprócz testów negatywnych.
Rzeczywisty rezultat	W metodach GET można zaobserwować procent Error w przedziale 3-10%. Niektóre wywołania metody GET zwracają kod błędu 500 oraz wyjątek ConcurrentModificationException.

Aby wyeliminować racing conditions w metodach get zostały wykorzystane inne struktury danych, MutableMapOf oraz MutableList zostały zamienione na ConcurrentHashMap oraz CopyOnWriteArrayList a retesty potwierdziły skuteczność wprowadzonych zmian.

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
[GET] Get package list under higher network traffic	1000	1	0	269	9.63	0.00%	101.8/sec	4702.01	15.12	47281.9
[GET] Get package list under stress testing army group	6250	2	0	253	8.74	0.00%	2062.7/sec	43919.24	306.18	21803.1
[GET] Get package list under small network traffic	100	4	0	270	26.81	0.00%	22.2/sec	631.68	3.29	29191.8
[POST] Adding packages with wrong types of data in ...	350	7	0	257	22.04	100.00%	179.8/sec	43.89	61.75	250.0
[POST] Correctly add package under higher network t...	500	3	0	264	16.40	0.00%	102.0/sec	29.65	36.52	297.8
[GET] Get form list under stress testing army group	6250	2	0	136	6.37	0.00%	2250.6/sec	47988.36	334.08	21833.9
[GET] Get forms list under higher network traffic	1000	1	0	62	3.67	0.00%	104.7/sec	4836.67	15.54	47303.7
[GET] Try to get package list as unauthorized user	6250	1	0	92	5.35	100.00%	2253.9/sec	460.02	341.16	209.0
[POST] Correctly add form under higher network traffic	500	2	0	131	10.58	0.00%	107.7/sec	34.67	43.83	329.6
[GET] Get forms list under small network traffic	100	3	0	85	8.98	0.00%	23.6/sec	836.85	3.43	36385.6
TOTAL	22300	2	0	270	8.13	29.60%	2270.9/sec	37350.22	369.71	16842.2